RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AI23231-PRINCIPLES OF ARTIFICIAL INTELLIGENCE LAB

[Regulation 2023]

II Year/III Semester

LABORATORY MANUAL

RAJALAKSHMI ENGINEERING COLLEGE

VISION & MISSION

OF

RAJALAKSHMI ENGINEERING COLLEGE

Vision

To be an institution of excellence in Engineering, Technology and Management Education & Research. To provide competent and ethical professionals with a concern for society.

Mission

To impart quality technical education imbibed with proficiency and humane values. To provide right ambience and opportunities for the students to develop into creative, talented and globally competent professionals. To promote research and development in technology and management for the benefit of the society.

RAJALAKSHMI ENGINEERING COLLEGE

B. E. ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VISION:

- To promote highly Ethical and Innovative Computer Professionals through excellence in teaching, training and research.

MISSION:

- To produce globally competent professionals, motivated to learn the emerging technologies and to be innovative in solving real world problems.
- To promote research activities amongst the students and the members of faculty that could benefit the society.
- To impart moral and ethical values in their profession.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: Graduates will demonstrate their technical skills and competency in various applications through the use of Artificial Intelligence and Data Science.

PEO 2: To produce motivated graduates with capability to apply acquired knowledge and skills in data analytics and visualization to develop viable systems.

PEO 3: Graduates will establish their knowledge by adopting Artificial Intelligence and Data Science technologies to solve the real world problems

PEO 4: To produce graduates with potential to participate in life-long learning through professional developments for societal needs with ethical values.

PROGRAMME OUTCOMES (POs)

PO1: Engineering knowledge: Apply the knowledge of Mathematics, Science, Engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO 6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

A graduate of the Artificial Intelligence and Data Science Learning Program will demonstrate.

PSO 1: Foundation Skills: Apply computing theory, languages and algorithms, as well as mathematical and statistical models, and the principles of optimization to appropriately formulate and use data analysis.

PSO 2: Problem-Solving Skills: The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for business automation.

PSO 3: Successful Progression: Ability to critique the role of information and analytics for an innovative career, research activities and consultancy.

Syllabus

Subject Code	Subject Name(Lab oriented Theory Courses)	Category	L	T	P	C
AI23231	PRINCIPLES OF ARTIFICIAL INTELLIGENCE	PC	3	0	2	4

Objectives:

- To Understand the various characteristics of a problem solving agent
- To Learn about the different strategies involved in problem solving.
- To Learn about solving problems with various constraints.
- To Learn about various knowledge representation
- To Understand the different models of reasoning and decision making

UNIT-I	INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND PROBLEM-SOLVING AGENT	9

AI-Introduction. Intelligent Agents, Agents & environment, nature of environment, structure of agents, goal-based agents, utility-based agents, learning agents. Defining the problem as state space search, production system, problem characteristics, issues in the design of search programs.

UNIT-II	SEARCH TECHNIQUES	9

Problem solving agents, searching for solutions; uniform search strategies: breadth first search, depth first search, depth limited search, bidirectional search. Heuristic search strategies Greedy best-first search, A* search, AO* search, memory bounded heuristic search: local search algorithms & optimization problems: Hill climbing search, simulated annealing search, local beam search.

UNIT-III	CONSTRAINT SATISFACTION PROBLEMS AND GAME THEORY	9

Local search for constraint satisfaction problems. Adversarial search, Games, optimal decisions & strategies in games, the min max search procedure, alpha-beta pruning.

UNIT-IV	KNOWLEDGE REPRESENTATION	9

AI for knowledge representation, rule-based knowledge representation, procedural and declarative knowledge, Logic programming, Forward and backward reasoning.			
UNIT-V	REASONING & DECISION MAKING		9
Statistical Reasoning: Probability and Bays' Theorem, Certainty Factors and Rule-Base Systems, Bayesian Networks, Dempster-Shafer Theory, Fuzzy Logic. Decision networks, Markov Decision Process. Expert System			
	Contact Hours	:	45

List of Experiments	
1	Programs on Problem Solving a. Write a program to solve 8 Queens problem b. Solve any problem using depth first search c. Implement MINIMAX algorithm d. Implement A* algorithm
2	Programs on Decision Making and Knowledge Representation a. Introduction to PROLOG i) Find minimum maximum of two numbers ii) Here are some simple clauses. likes(mary,food). likes(mary,wine). likes(john,wine). likes(john,mary). The following queries yield the specified answers. | ?- likes(mary,food). yes. | ?- likes(john,wine). yes.

	| ?- likes(john,food). no. How do you add the following facts? 1. John likes anything that Mary likes 2. John likes anyone who likes wine 3. John likes anyone who likes themselves
	b. Implementation of Unification and Resolution Algorithm
	c. Implementation of Backward Chaining
	d. Implementation of Forward Chaining
3	Programs on Planning and Learning a. Implementation of Blocks World program
	b. Implementing a fuzzy inference system

Requirements	
Hardware	Intel i3, CPU @ 1.20GHz 1.19 GHz, 4 GB RAM, 32 Bit Operating System
Software	C or Python Knowledge representation experiments can be performed using a PROLOG TOOL
Operating System	Windows

AI23231 – ARTIFICIAL INTELLIGENCE LAB PLAN

Exercise No.	Exercise Name	Required Hours
1	Write a program to solve 8 Queens problem	2
2	Solve any problem using depth first search	2
3	Implementation of MINIMAX algorithm	2
4	Implementation of A* algorithm	2
5	Find minimum maximum of two numbers	1
6	Implementation of Simple Clause likes(mary,food). likes(mary,wine). likes(john,wine). likes(john,mary). The following queries yield the specified answers. | ?- likes(mary,food). yes. | ?- likes(john,wine). yes. | ?- likes(john,food). no. How do you add the following facts? 1. John likes anything that Mary likes 2. John likes anyone who likes wine 3. John likes anyone who likes themselves	1
7	Implementation of Unification and Resolution Algorithm	2
8	Implementation of Backward Chaining	2
9	Implementation of Forward Chaining	2
10	Implementation of Blocks World program	2
11	Implementing a fuzzy inference system	2

Course Outcomes (COs)

Course Name: Artificial Intelligence Lab **Course Code: AI23231**

Outcome 1	Basic knowledge representation, problem solving, and learning methods of artificial intelligence.
Outcome 2	Provide the apt agent strategy to solve a given problem
Outcome 3	Represent a problem using first order and predicate logic
Outcome 4	Design applications like expert systems and chat-bot.
Outcome 5	Suggest the different models of reasoning and decision making for any given problem

CO-PO –PSO matrices of course

PO/PSO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CS23231.1	3	3	1	-	2	1	1	1	1	-	2	1	2	1	1
CS23231.2	2	2	1	-	2	1	2	-	-	-	2	2	1	1	1
CS23231.3	3	3	1	-	3	-	1	-	-	-	3	1	2	3	2
CS23231.4	2	2	2	2	3	-	1	2	-	-	2	2	2	2	2
CS23231.5	2	3	-	-	2	1	1	1	-	-	2	2	2	2	2
Average	2.4	2.4	1.8	2.0	2.4	0.6	1.2	0.8	0.2	-	2.0	1.8	1.8	1.8	1.6

Note: Enter correlation levels 1,2 or 3 as defined below:

1: Slight(Low) 2: Moderate(Medium) 3:Substantial(High)If there is no correlation, put"-"

CO-PO-PSO JUSTIFICATION

CO-PO/PSO	Level	Justification
CO1-PO1	3	Strongly correlated as CO1 builds foundational knowledge in AI and its methodologies.
CO1-PO2	3	Strongly supports problem analysis, as AI techniques are used to solve problems efficiently.
CO1-PO3	1	Weak correlation since design and development of AI models is not the primary focus.
CO1-PO4	-	No correlation
CO1-PO5	2	Moderate correlation as some modern AI tools are introduced but not deeply covered.
CO1-PO6	1	Weak correlation as sustainability aspects are not the primary focus.
CO1-PO7	1	Weak correlation because ethical considerations are minor.
CO1-PO8	1	Weak correlation due to limited societal impact.
CO1-PO9	1	Weak correlation due to limited focus on team work
CO1-P10	-	No correlation
CO1-P11	2	Moderate correlation as project management skills are involved.
CO1-P12	1	Weak correlation with lifelong learning since basic AI concepts form a foundation for further study.
CO1-PSO1	2	Moderate correlation with domain-specific AI knowledge and applications.
CO1-PSO2	1	Weak correlation due to limited practical application.
CO1-PSO3	1	Weak correlation since applied AI techniques are only slightly covered.
CO2-PO1	2	Moderate correlation, as AI agents require fundamental problem-solving techniques.
CO2-PO2	2	Moderate correlation with analytical skills, as strategies are predefined.
CO2-PO3	1	Moderate correlation with design and development.
CO2-PO4	-	No correlation with investigation aspects.
CO2-PO5	2	Moderate correlation since engineering tools are used.

CO2-PO6	1	Weak correlation due to minimal environmental considerations.
CO2-PO7	2	Moderate correlation as ethical factors play a role.
CO2-PO8	-	No correlation
CO2-PO9	-	No correlation
CO2-P10	-	No correlation
CO2-P11	2	Moderate correlation with modern tools.
CO2-P12	2	Moderate correlation with lifelong learning since AI strategies evolve with time.
CO2-PSO1	1	Weak correlation as intelligent agent strategies align with industry AI practices.
CO2-PSO2	1	Weak correlation, as domain-specific applications are not deeply explored.
CO2-PSO3	1	Weak Correlation as students can limit building effective AI-based solutions.
CO3-PO1	3	Strong correlation, as logical representation is fundamental in AI.
CO3-PO2	3	Strong correlation with problem analysis using formal logic methods.
CO3-PO3	1	Weak correlation, as problem representation does not involve design aspects.
CO3-PO4	-	No correlation
CO3-PO5	3	Strong correlation, as formal logic is applied using AI-based tools.
CO3-PO6	-	No correlation
CO3-PO7	1	Weak correlation with ethics.
CO3-PO8	-	No correlation
CO3-PO9	-	No correlation
CO3-P10	-	No correlation
CO3-P11	3	Strong correlation with modern tools and applications
CO3-PO12	1	Weak correlation with lifelong learning since logic representation is foundational.
CO3-PSO1	2	Strong correlation with domain-specific AI logic and problem representation techniques.

CO3-PSO2	3	Moderate correlation, as logic plays a role in applied AI applications.
CO3-PSO3	2	Moderate correlation with students gain the skills needed to design and develop AI-based solutions for real-world challenges.
CO4-PO1	2	Moderate correlation, as AI knowledge is applied to real-world applications.
CO4-PO2	2	Moderate correlation with problem analysis and designing intelligent systems.
CO4-PO3	2	Moderate correlation, as AI systems require structured design methodologies.
CO4-PO4	2	Investigative methods are employed to test and analyze graph algorithms for correctness and efficiency.
CO4-PO5	3	Strong correlation, as AI development involves the use of modern tools and frameworks.
CO4-PO6	-	No correlation
CO4-PO7	1	Moderate correlation, as ethical concerns in AI applications are addressed.
CO4-PO8	2	By understanding knowledge representation and its ethical implications, students develop AI solutions that align with professional ethics and societal responsibilities
CO4-PO9	-	No correlation
CO4-PO10	-	No correlation
CO4-PO11	2	Moderate correlation, as project management is needed for AI system development.
CO4-P12	2	Students must continuously update their knowledge, learn new technologies, and adapt to advancements in AI, such as natural language processing, deep learning, and ethical AI principles.
CO4-PSO1	2	Moderate correlation with technical skills required for AI-based applications.
CO4-PSO2	2	Moderate correlation with domain-specific AI applications.
CO4-PSO3	2	Moderate correlation with industry-focused AI system development.
CO5-PO1	2	Strong correlation, as decision-making models are fundamental AI techniques.
CO5-PO2	3	Strong correlation with problem-solving and analysis in AI.
CO5-PO3	-	No correlation
CO5-PO4	-	No correlation

CO5-PO5	2	Moderate correlation, as AI reasoning models are applied using modern tools.
CO5-PO6	1	By learning how AI systems reason and make decisions, students can analyze the societal and ethical impacts of AI solutions, ensuring they are fair, safe, and beneficial to society.
CO5-PO7	1	Weak correlation, as ethical concerns in decision-making are indirectly involved.
CO5-PO8	1	students can design AI systems that follow ethical guidelines, avoid bias, ensure transparency, and uphold professional responsibilities
CO5-PO9	-	No correlation
CO5-PO10	-	No correlation
CO5-PO11	2	students can optimize resources, assess risks, and make informed financial and managerial decisions, ensuring the successful execution of AI projects
CO5-P12	2	Moderate correlation with lifelong learning since AI reasoning models evolve over time.
CO5-PSO1	2	Moderate correlation with domain-specific AI decision-making techniques.
CO5-PSO2	2	Moderate correlation, as decision-making models are used in various AI applications.
CO5-PSO3	2	Moderate correlation with industry-focused AI solutions.

Faculty-in-charge **Mentor** **HoD**

Ex No: 1

IMPLEMENT EIGHT QUEENS PROBLEM

Aim:

To Implement Eight Queens Problem using backtracking

Procedure:

Program

```
N = 8  # Size of the chessboard (8x8)

# Function to print the solution
def printSolution(board):
    for row in board:
        for i in range(N):
            print("Q" if row[i] else ".", end=" ")
        print()
    print()  # Add a newline for readability

# Function to check if a queen can be placed on board[row][col]
def isSafe(board, row, col):
    # Check the column
    for i in range(row):
        if board[i][col]:
            return False
```

```
    # Check the upper left diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j]:
            return False

    # Check the upper right diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, N)):
        if board[i][j]:
            return False
    return True

# Function to solve the 8 Queens problem using backtracking
def solve(board, row):
    if row == N:
        printSolution(board)
        return True  # Stop after finding one solution

    for col in range(N):
        if isSafe(board, row, col):
            board[row][col] = 1  # Place queen

            if solve(board, row + 1):  # Recur to place next queen
                return True  # Stop after first valid solution

            board[row][col] = 0  # Backtrack (remove queen)

    return False

# Main function to initialize the board and start solving the problem
def eightQueens():
    board = [[0 for _ in range(N)] for _ in range(N)]
```

```
    if solve(board, 0):
        print("One possible solution is:")
    else:
        print("Solution does not exist")

# Calling the function
eightQueens()
```

Output:

```
One of the possible solutions is:
. . . . Q . . .
. . . Q . . . .
. . . . . Q . .
Q . . . . . . .
. . . . . . Q .
. Q . . . . . .
. . Q . . . . .
. . . . . . . Q
```

Ex No: 1b

IMPLEMENTATION OF DEPTH FIRST SEARCH

Aim:

To implement depth first search

Procedure:

Program:

```
# DFS Algorithm Implementation
class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.graph = {i: [] for i in range(vertices)}

    # Function to add an edge to the graph
    def add_edge(self, u, v):
        self.graph[u].append(v)
        self.graph[v].append(u)

    # Recursive function for DFS traversal
    def dfs(self, v, visited):
        print(v, end=" ")  # Print the current node
        visited[v] = True  # Mark the node as visited
```

```
        # Recur for all the adjacent vertices
        for neighbor in self.graph[v]:
            if not visited[neighbor]:
                self.dfs(neighbor, visited)

# Main function to demonstrate DFS
def depth_first_search():
    # Creating a graph with 6 vertices
    g = Graph(6)
        # Adding edges
    g.add_edge(0, 1)
    g.add_edge(0, 2)
    g.add_edge(1, 3)
    g.add_edge(1, 4)
    g.add_edge(2, 5)
    # To track visited nodes
    visited = [False] * g.vertices

    # Start DFS traversal from vertex 0
    print("Depth First Search starting from vertex 0:")
    g.dfs(0, visited)

# Calling the main function
depth_first_search()
```

Output:

```
Depth First Search starting from vertex 0:
0 1 3 4 2 5
```

Ex No: 1c

IMPLEMENTATION OF MINIMAX algorithm

Aim*:*

To implement MINIMAX algorithm.

Procedure:

Program:

```
# Constants for players
PLAYER_X = 1
PLAYER_O = -1
EMPTY = 0

# Evaluate the board
def evaluate(board):
    for row in range(3):
        if board[row][0] == board[row][1] == board[row][2] != EMPTY:
            return board[row][0]
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col] != EMPTY:
            return board[0][col]
    if board[0][0] == board[1][1] == board[2][2] != EMPTY:
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] != EMPTY:
```

```
        return board[0][2]
    return 0

# Check if moves are left
def isMovesLeft(board):
    for row in range(3):
        for col in range(3):
            if board[row][col] == EMPTY:
                return True
    return False

# Minimax function
def minimax(board, isMax):
    score = evaluate(board)
    if score == PLAYER_X: return score
    if score == PLAYER_O: return score
    if not isMovesLeft(board): return 0

    if isMax:
        best = -float('inf')
        for row in range(3):
            for col in range(3):
                if board[row][col] == EMPTY:
                    board[row][col] = PLAYER_X
                    best = max(best, minimax(board, not isMax))
                    board[row][col] = EMPTY
        return best
    else:
        best = float('inf')
        for row in range(3):
            for col in range(3):
```

```
            if board[row][col] == EMPTY:
                board[row][col] = PLAYER_O
                best = min(best, minimax(board, not isMax))
                board[row][col] = EMPTY
        return best

# Find the best move for PLAYER_X
def findBestMove(board):
    bestVal = -float('inf')
    bestMove = (-1, -1)
    for row in range(3):
        for col in range(3):
            if board[row][col] == EMPTY:
                board[row][col] = PLAYER_X
                moveVal = minimax(board, False)
                board[row][col] = EMPTY
                if moveVal > bestVal:
                    bestMove = (row, col)
                    bestVal = moveVal
    return bestMove

# Print the board
def printBoard(board):
    for row in board:
        print(" ".join(["X" if x == PLAYER_X else "O" if x == PLAYER_O else "." for x in
row]))

# Example game
board = [
    [PLAYER_X, PLAYER_O, PLAYER_X],
    [PLAYER_O, PLAYER_X, EMPTY],
```

```
    [EMPTY, PLAYER_O, PLAYER_X]
]

print("Current Board:")
printBoard(board)

move = findBestMove(board)
print(f"Best Move: {move}")
board[move[0]][move[1]] = PLAYER_X
print("\nBoard after best move:")
printBoard(board)
```

Output:

```
Current Board:
X O X
O X .
. O X

Best Move: (2, 0)

Board after best move:
X O X
O X .
X O X
```

Ex No: 1d

IMPLEMENTATION OF A* SEARCH ALGORITHM

Aim:

To implement A* Search Algorithm

Procedure:

Program:

```
import heapq

class Node:
    def __init__(self, state, parent, g, h):
        self.state = state
        self.parent = parent
        self.g = g  # Cost to reach the current node
        self.h = h  # Heuristic cost from current node to goal
        self.f = g + h  # Total cost (f = g + h)

    def __lt__(self, other):
        return self.f < other.f
def a_star_search(start, goal, heuristic, neighbors):
    open_list = []
    closed_list = set()

    start_node = Node(start, None, 0, heuristic(start, goal))
    heapq.heappush(open_list, start_node)
```

```
    while open_list:
        current_node = heapq.heappop(open_list)

        if current_node.state == goal:
            # Reconstruct the path from start to goal
            path = []
            while current_node:
                path.append(current_node.state)
                current_node = current_node.parent
            return path[::-1]

        closed_list.add(current_node.state)

        for neighbor, cost in neighbors(current_node.state):
            if neighbor in closed_list:
                continue

            g = current_node.g + cost
            h = heuristic(neighbor, goal)
            neighbor_node = Node(neighbor, current_node, g, h)

            if not any(open_node.state == neighbor and open_node.f <= neighbor_node.f for
open_node in open_list):
                heapq.heappush(open_list, neighbor_node)

    return None  # If no path is found
# Example of usage

def heuristic(state, goal):
    # Simple Manhattan distance heuristic for a grid (assuming coordinates)
    return abs(state[0] - goal[0]) + abs(state[1] - goal[1])
```

```
def neighbors(state):
    # Define neighbors as adjacent grid cells (up, down, left, right)
    x, y = state
    return [
        ((x + 1, y), 1),  # Right
        ((x - 1, y), 1),  # Left
        ((x, y + 1), 1),  # Up
        ((x, y - 1), 1)   # Down
    ]


start = (0, 0)
goal = (3, 3)

path = a_star_search(start, goal, heuristic, neighbors)
print("Path from start to goal:", path)
```

Output:

For `start = (0, 0)` and `goal = (3, 3)`:

Path from start to goal: [(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (3, 2), (3, 3)]

Ex No: 2

IMPLEMENTATION OF DECISION MAKING AND KNOWLEDGE REPRESENTATION

Aim:

To implement decision making and knowledge representation using prolog tool.

Prolog Code:

% Rule to find the minimum of two numbers
minimum(X, Y, X) :- X =< Y. % If X is less than or equal to Y, X is the minimum.
minimum(X, Y, Y) :- X > Y. % If X is greater than Y, Y is the minimum.

% Rule to find the maximum of two numbers maximum(X, Y, X) :- X >= Y. % If X is greater than or equal to Y, X is the maximum. maximum(X, Y, Y) :- X < Y. % If X is less than Y, Y is the maximum.

Example Queries:

1. To find the minimum of two numbers:

```
?- minimum(5, 10, Min).
```

Output:

Min = 5.

2. To find the maximum of two numbers:

?- maximum(5, 10, Max).

Output:

Max = 10.

?- minimum(8, 3, Min), maximum(8, 3, Max).

Output:

```
Min = 3, Max = 8.
```

Prolog Code:

```
% Given facts
likes(mary, food).
likes(mary, wine).
likes(john, wine).
likes(john, mary).

% Rules based on the conditions:
likes(john, X) :- likes(mary, X).  % John likes anything that Mary likes
likes(john, Y) :- likes(Y, wine).  % John likes anyone who likes wine
likes(john, Y) :- likes(Y, Y).     % John likes anyone who likes themselves

% Sample queries:
% Query 1: Does John like food?
% ?- likes(john, food).

% Query 2: Does John like wine?
% ?- likes(john, wine).

% Query 3: Does John like food if Mary likes food?
% ?- likes(john, food).

% Query 4: Who does John like?
% ?- likes(john, Y).
```

Output:

Query: `?- likes(john, food).`

```
yes
```

Query: `?- likes(john, wine).`

```
yes
```

Query: `?- likes(john, food).`

```
yes
```

Query: `?- likes(john, Y).`

```
Y = mary ;
Y = john ;
Y = wine ;
```

Query?- likes(john, Y).

Y = mary ;

Y = john ;

Y = wine ;

Ex No: 2b

IMPLEMENTATION OF UNIFICATION AND RESOLUTION ALGORITHM

Aim:

To implement unification and resolution algorithm using python.

Procedure:

Program:

```
# Unification and Resolution in Python

def unify(var1, var2, subst):
    """Unify two variables or terms with substitution."""
    if subst is None:
        return None
    elif var1 == var2:
        return subst
    elif isinstance(var1, str) and var1.islower():
        return unify_var(var1, var2, subst)
    elif isinstance(var2, str) and var2.islower():
        return unify_var(var2, var1, subst)
    elif isinstance(var1, list) and isinstance(var2, list) and len(var1) == len(var2):
        return unify(var1[1:], var2[1:], unify(var1[0], var2[0], subst))
    else:
        return None
```

```
def unify_var(var, x, subst):
    """Handle unification for variables."""
    if var in subst:
        return unify(subst[var], x, subst)
    elif x in subst:
        return unify(var, subst[x], subst)
    elif occurs_check(var, x, subst):
        return None
    else:
        subst[var] = x
        return subst

def occurs_check(var, x, subst):
    """Check for cyclic dependencies in substitution."""
    if var == x:
        return True
    elif isinstance(x, list):
        return any(occurs_check(var, xi, subst) for xi in x)
    elif x in subst:
        return occurs_check(var, subst[x], subst)
    return False

def resolution(kb, query):
    """Apply resolution algorithm to check satisfiability."""
    clauses = kb + [negate(query)]
    while True:
        new_clauses = []
        for i, ci in enumerate(clauses):
            for j, cj in enumerate(clauses):
                if i >= j:
```

```
                continue
            resolvents = resolve(ci, cj)
            if [] in resolvents:
                return True  # Query is satisfiable
            for res in resolvents:
                if res not in new_clauses:
                    new_clauses.append(res)
        if all(new in clauses for new in new_clauses):
            return False  # Query is not satisfiable
        clauses.extend(new_clauses)

def negate(literal):
    """Negate a literal or a list of literals."""
    if isinstance(literal, list):
        return [negate(lit) for lit in literal]
    if literal.startswith("~"):
        return literal[1:]
    else:
        return f"~{literal}"

def resolve(clause1, clause2):
    """Resolve two clauses to find resolvents."""
    resolvents = []
    for lit1 in clause1:
        for lit2 in clause2:
            if lit1 == negate(lit2):
                new_clause = list(set(clause1 + clause2))
                new_clause.remove(lit1)
                new_clause.remove(lit2)
                if new_clause not in resolvents:
                    resolvents.append(new_clause)
```

```
    return resolvents
if __name__== "__main__":
    # Example knowledge base and query
    knowledge_base = [
        ["~P", "Q"],
        ["P"],
        ["~Q", "R"],
        ["~R"]
    ]
    query = ["R"]  # Representing the query as a list
    print("Knowledge Base:", knowledge_base)
    print("Query:", query)
    result = resolution(knowledge_base, query)
    if result:
        print("The query is satisfiable.")
    else:
        print("The query is not satisfiable.")
```

Output:

```
Knowledge Base:
[
    ["~P", "Q"],
    ["P"],
    ["~Q", "R"],
    ["~R"]
]
Query:
["R"]
Knowledge Base: [['~P', 'Q'], ['P'], ['~Q', 'R'], ['~R']]
Query: ['R']
The query is not satisfiable.
```

Ex No: 2c

IMPLEMENTATION OF BACKWARD CHAINING

Aim*:*

To implement backward chaining.

Procedure:

Program:

```
# Facts: The knowledge base containing information about facts
facts = {
    'a': True,
    'b': True,
    'c': False
}
# Rules: List of rules where each rule is a tuple (conclusion, conditions)
rules = [
    ('d', ['a', 'b']),  # d can be concluded if a and b are true
    ('e', ['b', 'c']),  # e can be concluded if b and c are true
    ('f', ['d', 'e'])  # f can be concluded if d and e are true
]
# Backward Chaining Function
def backward_chaining(goal, facts, rules):
    # Check if the goal is already a fact
    if goal in facts:
        return facts[goal]
```

```
        # Check if the goal can be concluded from any rule
    for rule in rules:
        head, body = rule
        if head == goal:
            # Check if all the conditions in the body of the rule are true
            if all(backward_chaining(fact, facts, rules) for fact in body):
                return True

    # If the goal cannot be proved
    return False

# Example Usage:
goal = 'f'

if backward_chaining(goal, facts, rules):
    print(f"The goal '{goal}' can be achieved.")
else:
    print(f"The goal '{goal}' cannot be achieved.")
```

Output:

Expected Output for the Goal `'f'`:
The goal 'f' cannot be achieved.

Expected Output for the Goal `'d'`:
The goal 'd' can be achieved.

Ex No: 2d

IMPLEMENTION OF FORWARD CHAINING

Aim:

To implement forward Chaining.

Procedure:

Program:

```
# Facts: The knowledge base containing information about facts
facts = {
    'a': True,
    'b': True,
    'c': False
}

# Rules: List of rules where each rule is a tuple (conclusion, conditions)
rules = [
    ('d', ['a', 'b']),  # d can be concluded if a and b are true
    ('e', ['b', 'c']),  # e can be concluded if b and c are true
    ('f', ['d', 'e'])  # f can be concluded if d and e are true
]

# Forward Chaining Function
def forward_chaining(facts, rules, goal):
```

```
    inferred = set(facts.keys())  # Initialize inferred facts with the given facts
    new_inferred = set(facts.keys())  # Start with the facts
        while new_inferred:
        current_inferred = set()  # Set to hold newly inferred facts
        for rule in rules:
            head, body = rule
            if head not in inferred and all(fact in inferred for fact in body):
                current_inferred.add(head)
                if current_inferred:
            inferred.update(current_inferred)
            new_inferred = current_inferred  # Set new inferred facts for the next iteration
        else:
            new_inferred = set()  # No new facts can be inferred

    return goal in inferred  # Check if the goal is in the inferred facts

# Example Usage
goal = 'f'

if forward_chaining(facts, rules, goal):
    print(f"The goal '{goal}' can be achieved.")
else:
    print(f"The goal '{goal}' cannot be achieved.")
```

Output:

The goal 'f' can be achieved.

The goal 'e' cannot be achieved.

The goal 'd' can be achieved.

Ex No: 3a

IMPLEMENTATION OF BLOCKS WORLD PROGRAM

Aim:

To implement binary search tree.

Procedure:

Program:

```
# Class representing the state of the blocks world
class BlocksWorld:
    def __init__(self, num_blocks):
        # Initialize blocks world with num_blocks on the table
        self.state = [[block] for block in range(num_blocks)]  # Each block starts on the table
        self.num_blocks = num_blocks

    def display_state(self):
        # Display the current state of the blocks world
        for stack in self.state:
            print(f"Block(s) on stack: {stack}")

    def move(self, block, destination):
        # Move block onto another block or the table
        # Find the block in the current state and move it
        source_stack = self.find_block(block)
```

```
        destination_stack = self.find_block(destination)

        if source_stack is None or destination_stack is None:
            print(f"Invalid block {block} or destination {destination}.")
            return

        # Remove the block from its current position
        source_stack.remove(block)

        # Put the block on the destination stack
        destination_stack.append(block)

        # Display the state after the move
        self.display_state()

    def find_block(self, block):
        # Find the stack that contains the given block
        for stack in self.state:
            if block in stack:
                return stack
        return None

    def goal_state(self, goal):
        # Goal state is a list of stacks that represents the desired configuration
        self.state = goal
        print("Goal state set.")
        self.display_state()

# Main program to solve Blocks World problem
def main():
    # Initialize the Blocks World with 3 blocks (0, 1, 2)
```

```
    blocks_world = BlocksWorld(3)

    print("Initial state:")
    blocks_world.display_state()

    # Define the goal state (Block 0 is on top of Block 1, and Block 2 is on the table)
    goal = [[0, 1], [2]]  # Goal configuration
    blocks_world.goal_state(goal)

    print("\nPerforming Moves:")
    # Move Block 0 onto Block 2
    blocks_world.move(0, 2)

    # Move Block 1 onto Block 2
    blocks_world.move(1, 2)

    # Move Block 2 onto the table (empty stack)
    blocks_world.move(2, 0)

# Run the main program
if __name___== "__main__":
    main()
```

Output:

Initial state:

Block(s) on stack: [0]

Block(s) on stack: [1]

Block(s) on stack: [2]

Goal state set.

Block(s) on stack: [0, 1]

Block(s) on stack: [2]

Performing Moves:

Block(s) on stack: [1]

Block(s) on stack: [2, 0]

Block(s) on stack: []

Block(s) on stack: [1]

Block(s) on stack: [2, 0]

Block(s) on stack: []

Block(s) on stack: [1, 2]

Block(s) on stack: [0]

Block(s) on stack: []

Ex No: 3b

IMPLEMENTION OF A FUZZY INFERENCE SYSTEM

Aim*:*

To implement Fuzzy Inference System.

Procedure:

Program:

Example: Simple Fuzzy Inference System for Temperature Control

```
import numpy as np
import skfuzzy as fuzz
import matplotlib.pyplot as plt

# Step 1: Define the fuzzy variables (input: temperature, output: fan speed)
temperature = np.arange(0, 41, 1)  # Temperature from 0 to 40 degrees Celsius
fan_speed = np.arange(0, 11, 1)  # Fan speed from 0 to 10

# Step 2: Define fuzzy membership functions
# Temperature membership functions: Low, Medium, High
temp_low = fuzz.trimf(temperature, [0, 0, 20])
temp_medium = fuzz.trimf(temperature, [10, 20, 30])
```

```
temp_high = fuzz.trimf(temperature, [20, 30, 40])

# Fan speed membership functions: Low, Medium, High
fan_low = fuzz.trimf(fan_speed, [0, 0, 5])
fan_medium = fuzz.trimf(fan_speed, [2, 5, 8])
fan_high = fuzz.trimf(fan_speed, [5, 10, 10])

# Step 3: Visualize the membership functions
plt.figure(figsize=(10, 6))

plt.subplot(2, 1, 1)
plt.plot(temperature, temp_low, label='Low')
plt.plot(temperature, temp_medium, label='Medium')
plt.plot(temperature, temp_high, label='High')
plt.title("Temperature Membership Functions")
plt.xlabel("Temperature (°C)")
plt.ylabel("Membership Degree")
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(fan_speed, fan_low, label='Low')
plt.plot(fan_speed, fan_medium, label='Medium')
plt.plot(fan_speed, fan_high, label='High')
plt.title("Fan Speed Membership Functions")
plt.xlabel("Fan Speed")
plt.ylabel("Membership Degree")
plt.legend()

plt.tight_layout()
plt.show()
```

```
# Step 4: Fuzzification - Define a crisp temperature input
temperature_input = 28  # Example temperature

# Fuzzify the input
temp_low_level = fuzz.interp_membership(temperature, temp_low, temperature_input)
temp_medium_level = fuzz.interp_membership(temperature, temp_medium, temperature_input)
temp_high_level = fuzz.interp_membership(temperature, temp_high, temperature_input)

# Step 5: Apply fuzzy rules
# Rule 1: If temperature is Low, fan speed is Low
# Rule 2: If temperature is Medium, fan speed is Medium
# Rule 3: If temperature is High, fan speed is High

fan_low_level = temp_low_level
fan_medium_level = temp_medium_level
fan_high_level = temp_high_level

# Step 6: Defuzzification - Calculate the crisp output (fan speed)
fan_output = fuzz.defuzz(fan_speed, fan_low * fan_low_level + fan_medium * fan_medium_level + fan_high * fan_high_level, 'centroid')

# Step 7: Print the output
print(f"Temperature: {temperature_input}°C")
print(f"Fuzzified fan speed: {fan_output:.2f}")
```

Output

```
Temperature: 28°C
Fuzzified fan speed: 6.00
```

VIVA QUESTIONS WITH ANSWERS

1. What is Artificial Intelligence (AI)?

   Answer: Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think and act like humans.

2. What is an intelligent agent?

   Answer: An intelligent agent is a system that perceives its environment and takes actions to achieve its goals autonomously.

3. What are the types of environments an agent can operate in?

   Answer: Agents can operate in environments that are deterministic, observable, static, and dynamic, depending on the properties of the environment.

4. What is the difference between a reflex agent and a goal-based agent?

   Answer: A reflex agent acts based on current perceptions using predefined rules, while a goal-based agent evaluates actions to achieve specific goals.

5. What is a utility-based agent?

   Answer: A utility-based agent selects actions that maximize its expected utility or satisfaction based on available options.

6. What is the state space in AI?

   Answer: The state space is the set of all possible states that can be reached during the process of solving a problem.

7. What are the components of a production system?

   Answer: A production system consists of rules, a knowledge base, an inference engine, and working memory.

8. What is the difference between a well-defined problem and an ill-defined problem?

   Answer: A well-defined problem has clear objectives and constraints, while an ill-defined problem lacks clear goals and solutions.

9. What are the key issues in designing search programs?

   Answer: Key issues include time complexity, space complexity, optimality, completeness, and efficiency in solving a problem.

10. What is the role of fuzziness in fuzzy logic systems?

    Answer: Fuzziness in fuzzy logic allows for reasoning and decision-making with imprecise, uncertain, or approximate information.

11. What is a problem-solving agent?

    Answer: A problem-solving agent is an intelligent agent that takes actions to reach a desired goal from an initial state through a search process.

12. What is the Breadth-First Search (BFS) strategy?

    Answer: BFS is a search strategy that explores all possible nodes at the present depth level before moving on to nodes at the next depth level.

13. What is Depth-First Search (DFS)?

    Answer: DFS is a search strategy that explores as far down a branch of the tree as possible before backtracking to explore other branches.

14. What is Depth-Limited Search?

    Answer: Depth-Limited Search is a variation of DFS that limits the depth of search to a pre-defined depth level to prevent infinite loops.

15. What is Bidirectional Search?

    Answer: Bidirectional Search simultaneously searches from both the initial state and the goal state, meeting in the middle to find the solution more efficiently.

16. What is Greedy Best-First Search?

    Answer: Greedy Best-First Search selects the node that appears to be closest to the goal based on a heuristic function.

17. What is the A Search algorithm?*

    Answer: A* Search is a heuristic search algorithm that evaluates nodes based on both the cost to reach the node and the estimated cost to the goal, providing optimal solutions.

18. What is AO Search?*

    Answer: AO* Search is an algorithm used for solving problems in a Directed Acyclic Graph (DAG) by combining the concepts of A* and AND/OR graphs.

19. What is Hill Climbing Search?

    Answer: Hill Climbing Search is a local search algorithm that continuously moves towards the neighbor with the highest value, optimizing a specific objective function.

20. What is Simulated Annealing?

    Answer: Simulated Annealing is an optimization algorithm that explores the solution space by allowing occasional moves to worse solutions to escape local optima, inspired by the annealing process in metallurgy.

21. What is local search for constraint satisfaction problems (CSPs)?

    Answer: Local search for CSPs involves iteratively improving a candidate solution by making small changes to reduce constraint violations, aiming to find a solution that satisfies all constraints.

22. What is adversarial search in AI?

    Answer: Adversarial search is a process where agents compete against each other, and each agent aims to maximize its own benefit while minimizing the opponent's benefit, typically used in games like chess.

23. How are optimal decisions made in games?

    Answer: Optimal decisions in games are made by evaluating all possible moves and selecting the one that maximizes a player's chances of winning, considering both the current state and possible future moves.

24. What is the Min-Max search procedure?

    Answer: The Min-Max search procedure is an algorithm used in game theory to recursively determine the best move by assuming that the opponent will also play optimally, minimizing the maximizing player’s benefit.

25. What is Alpha-Beta pruning in adversarial search?

    Answer: Alpha-Beta pruning is an optimization technique for Min-Max search that eliminates branches of the game tree that will not affect the final decision, improving search efficiency.

26. . What is knowledge representation in AI?

    Answer: Knowledge representation in AI is the process of encoding information about the world in a form that a computer system can use to solve complex tasks like reasoning and problem-solving.

27. What is rule-based knowledge representation?

    Answer: Rule-based knowledge representation uses a set of "if-then" rules to encode knowledge, where each rule represents a relationship or condition about the world.

28. What is the difference between procedural and declarative knowledge?

    Answer: Procedural knowledge describes how to perform tasks or actions, while declarative knowledge represents facts or information about the world.

29. What is logic programming in AI?

    Answer: Logic programming is a programming paradigm where programs are written as sets of logical statements, and computation is performed through logical inference.

30. What is the difference between forward and backward reasoning?

    Answer: Forward reasoning starts from known facts and applies rules to reach a conclusion, while backward reasoning starts from a goal and works backward to find supporting facts.

31. What is Bayesian Probability and Bayes' Theorem?

    Answer: Bayesian Probability uses Bayes' Theorem to update the probability of a hypothesis based on new evidence, providing a method to calculate conditional probabilities.

32. What are Certainty Factors in Rule-Based Systems?

    Answer: Certainty factors quantify the degree of belief or confidence in a conclusion drawn from a rule-based system, allowing reasoning under uncertainty.

33. What are Bayesian Networks?

    Answer: Bayesian Networks are graphical models that represent probabilistic relationships between variables, where nodes represent variables and edges represent conditional dependencies.

34. What is the Dempster-Shafer Theory?

    Answer: The Dempster-Shafer Theory is a mathematical framework for reasoning with uncertainty, allowing the combination of evidence from different sources to assign belief functions to hypotheses.

35. What is a Markov Decision Process (MDP)?

    Answer: A Markov Decision Process is a mathematical model for decision-making where an agent interacts with an environment in discrete time steps, with outcomes determined by both the agent's actions and probabilistic transitions.